Onboarding - spring-boot-realworld-example-app

Table of Contents

Introduction

This repository implements a Spring Boot reference RealWorld example application. The codebase provides REST controllers, application services, domain model classes, MyBatis mappers/repositories, GraphQL datafetchers/mutations, and security configuration using JWT. The application configuration targets a local SQLite data file and includes custom Jackson serialization and MyBatis integration. The repository includes automated tests and a GitHub Actions workflow for CI.

Technical Summary

Aspect	Value
Language	Java (source files under java/)
Framework	Spring Boot (org.springframework.boot plugin and @SpringBootApplication)
Build System	Gradle (build.gradle present)

Java Version	11 (sourceCompatibility = '11' in build.gradle)
Database	SQLite (spring.datasource.url=jdbc:sqlite:dev.db in application.properties)
Persistence / Mapper	MyBatis (mybatis-spring-boot-starter dependency and mapper interfaces)
Security	Spring Security with JWT (spring-boot-starter-security dependency and WebSecurityConfig, JwtTokenFilter referenced)
GraphQL	Netflix DGS (graphql-dgs-spring-boot-starter dependency and graphql package)
API Documentation	OpenAPI / Swagger annotations present (io.swagger.v3.oas. annotations in controllers)
Testing	JUnit Platform, Spring Boot test, Rest-Assured (test dependencies in build.gradle)
CI/CD	GitHub Actions (.github/workflows/gradle.yml runs Gradle clean test)

Development Environment Requirements

- Gradle-based build (build.gradle present). The repository's CI uses ./gradlew clean test.
- Java 11 is declared as sourceCompatibility and targetCompatibility in build.gradle.
- Dependencies configured in build.gradle include Spring Boot starters, MyBatis, GraphQL DGS, Flyway, jjwt, Joda-Time, and sqlite-jdbc.
- The application configuration uses an embedded SQLite database file referenced as dev.db in src/main/resources/application.properties.
- Spotless with googleJavaFormat is configured in build.gradle for code formatting.
- No Docker or container files were found in the repository.

Local Setup and Execution

- The repository does not include an explicit, dedicated "run" or "setup" README with step-by-step local execution instructions. Use the build and configuration artifacts present as the source of truth.

- Evidence-backed commands and artifacts:
    - Build and run tests using the Gradle wrapper as used in CI:

```
./gradlew clean test
```

- Main application entry point: io.spring.RealWorldApplication with a standard SpringApplication.run(...) main method.
- Application configuration is in src/main/resources/application.properties. The datasource is configured to use SQLite at jdbc:sqlite:dev.db.
- The project configures a code generation task for GraphQL schema generation (generateJava task configured in build.gradle referencing src/main/resources /schema). Consult build.gradle for exact task properties.
- If you need to start the application locally, the repository does not provide an explicit scripted instruction. The Spring Boot plugin is applied in build.gradle (org. springframework.boot), and standard Gradle tasks (for example bootRun) may be available by convention; however, explicit run instructions are not provided in repository documentation.

Project Structure

The core Java sources are under a java/ hierarchy. Key packages and files (evidence-backed) are:

```
### java/
    ### io/
        ### spring/
            ### JacksonCustomizations.java
            ### MyBatisConfig.java
            ### RealWorldApplication.java
            ### Util.java
            ### api/
            #   ### ArticleApi.java
            #   ### ArticleFavoriteApi.java
            #   ### ArticlesApi.java
            #   ### CommentsApi.java
            #   ### CurrentUserApi.java
            #   ### ProfileApi.java
            #   ### TagsApi.java
            #   ### UsersApi.java
            #   ### exception/
```

```
#   #   ### CustomizeExceptionHandler.java
#   #   ### ErrorResource.java
#   #   ### ErrorResourceSerializer.java
#   #   ### FieldErrorResource.java
#   #   ### InvalidAuthenticationException.java
#   #   ### InvalidRequestException.java
#   #   ### NoAuthorizationException.java
#   #   ### ResourceNotFoundException.java
#   ### security/
#       ### JwtTokenFilter.java
#       ### WebSecurityConfig.java
### application/
#   ### ArticleQueryService.java
#   ### CommentQueryService.java
#   ### CursorPageParameter.java
#   ### CursorPager.java
#   ### DateTimeCursor.java
#   ### Node.java
#   ### Page.java
#   ### PageCursor.java
#   ### ProfileQueryService.java
#   ### TagsQueryService.java
#   ### UserQueryService.java
#   ### article/
#   #   ### ArticleCommandService.java
#   #   ### DuplicatedArticleConstraint.java
#   #   ### DuplicatedArticleValidator.java
#   #   ### NewArticleParam.java
#   #   ### UpdateArticleParam.java
#   ### data/
#   #   ### ArticleData.java
#   #   ### ArticleDataList.java
#   #   ### ArticleFavoriteCount.java
#   #   ### CommentData.java
#   #   ### ProfileData.java
#   #   ### UserData.java
#   #   ### UserWithToken.java
#   ### user/
#       ### DuplicatedEmailConstraint.java
#       ### DuplicatedEmailValidator.java
#       ### DuplicatedUsernameConstraint.java
#       ### DuplicatedUsernameValidator.java
#       ### RegisterParam.java
#       ### UpdateUserCommand.java
#       ### UpdateUserParam.java
#       ### UserService.java
### core/
#   ### article/
#   #   ### Article.java
```

```
#   #   ### ArticleRepository.java
#   #   ### Tag.java
#   ### comment/
#   #   ### Comment.java
#   #   ### CommentRepository.java
#   ### favorite/
#   #   ### ArticleFavorite.java
#   #   ### ArticleFavoriteRepository.java
#   ### service/
#   #   ### AuthorizationService.java
#   #   ### JwtService.java
#   ### user/
#       ### FollowRelation.java
#       ### User.java
#       ### UserRepository.java
### graphql/
#   ### ArticleDatafetcher.java
#   ### ArticleMutation.java
#   ### CommentDatafetcher.java
#   ### CommentMutation.java
#   ### MeDatafetcher.java
#   ### ProfileDatafetcher.java
#   ### RelationMutation.java
#   ### SecurityUtil.java
#   ### TagDatafetcher.java
#   ### UserMutation.java
#   ### exception/
#       ### AuthenticationException.java
#       ### GraphQLCustomizeExceptionHandler.java
### infrastructure/
    ### mybatis/
    #   ### DateTimeHandler.java
    #   ### mapper/
    #   #   ### ArticleFavoriteMapper.java
    #   #   ### ArticleMapper.java
    #   #   ### CommentMapper.java
    #   #   ### UserMapper.java
    #   ### readservice/
    #       ### ArticleFavoritesReadService.java
    #       ### ArticleReadService.java
    #       ### CommentReadService.java
    #       ### TagReadService.java
    #       ### UserReadService.java
    #       ### UserRelationshipQueryService.java
    ### repository/
    #   ### MyBatisArticleFavoriteRepository.java
    #   ### MyBatisArticleRepository.java
    #   ### MyBatisCommentRepository.java
    #   ### MyBatisUserRepository.java
```

```
                ### service/
                    ### DefaultJwtService.java
```

Codebase Orientation

The description below is conservative and limited to elements present in the repository.

- Controllers / API layer
    - Located under io.spring.api. Controllers expose REST endpoints for articles, comments, users, profiles, tags and favorites (for example ArticleApi, ArticlesApi, CommentsApi, UsersApi).
    - Controllers use Spring MVC annotations (GetMapping, PostMapping, RequestMapping, etc.) and include OpenAPI annotations.
- Application Services
    - Located under io.spring.application. Services provide query and command operations (ArticleQueryService, ArticleCommandService, UserService, CommentQueryService, TagsQueryService, ProfileQueryService).
    - Services interact with infrastructure read services and repositories to assemble DTOs and to perform business operations.
- Domain / Core
    - Domain entities and repository interfaces are under io.spring.core (article, comment, favorite, user, and service packages).
    - Core contains JwtService and AuthorizationService used by higher-level components.
- Infrastructure / Persistence
    - MyBatis mapper interfaces are under io.spring.infrastructure.mybatis.mapper (ArticleMapper, CommentMapper, UserMapper, ArticleFavoriteMapper).
    - Repository implementations using MyBatis are under io.spring.infrastructure.repository (MyBatisArticleRepository, MyBatisCommentRepository, MyBatisUserRepository, MyBatisArticleFavoriteRepository).
    - Read services for optimized queries are under io.spring.infrastructure.mybatis.readservice.

- GraphQL
    - GraphQL datafetchers and mutation handlers are under io.spring.graphql. build.gradle configures code generation for GraphQL schema under src /main/resources/schema.
- Configuration and Utilities
    - JacksonCustomizations.java registers a custom serializer for org.joda.time. DateTime.
    - MyBatisConfig.java enables transaction management.
    - WebSecurityConfig.java configures Spring Security, CORS, and registers JwtTokenFilter and PasswordEncoder beans.
- DTOs / Data classes
    - DTOs and data transfer objects are under io.spring.application.data and parameter objects under io.spring.application.article and io.spring. application.user.

Testing and Validation

- Test framework and dependencies (evidence from build.gradle):
    - JUnit Platform is enabled for the test task (useJUnitPlatform()).
    - testImplementation dependencies include org.springframework.boot:spring-boot-starter-test, org.mybatis.spring.boot:mybatis-spring-boot-starter-test, io.rest-assured artifacts, and spring-security-test.
- Test artifacts:
    - Tests are located under src/test/java with 23 detected test files (evidence list includes controller tests, service tests, repository tests and integration-style DbTestBase).
- Test execution:
    - CI runs tests using:

```
./gradlew clean test
```

- The project defines a clean task hook that deletes './dev.db' before cleaning (evidence in build.gradle).

API Exploration

- REST endpoints:
  - The repository includes controllers for the REST API. Detected endpoints (from controller evidence) include:
    - GET /articles/{slug}
    - PUT /articles/{slug}
    - DELETE /articles/{slug}
    - POST /articles/{slug}/favorite
    - DELETE /articles/{slug}/favorite
    - POST /articles
    - GET /articles/feed
    - GET /articles
    - POST /articles/{slug}/comments
    - GET /articles/{slug}/comments
    - DELETE /articles/{slug}/comments/{id}
    - GET /user
    - PUT /user
    - GET /profiles/{username}
    - POST /profiles/{username}/follow
    - DELETE /profiles/{username}/follow
    - GET /tags
- OpenAPI / Swagger:
  - Controllers contain io.swagger.v3.oas.annotations usage. This indicates the codebase contains OpenAPI/Swagger annotations, but the repository does not include an explicit generated OpenAPI document or a configured Swagger UI endpoint path in repository documentation.
- GraphQL:
  - GraphQL server elements exist under io.spring.graphql and build.gradle configures the com.netflix.dgs.codegen plugin with schemaPaths pointing to src/main/resources/schema. The repository includes GraphQL-related code (datafetchers and mutations).
- Security considerations:
  - WebSecurityConfig configures which REST endpoints are permitted anonymously and which require authentication. JwtTokenFilter is registered in the security filter chain. The JWT secret and sessionTime are specified in application.properties.

Working Guidelines

- Formatting and static style:

    - Spotless with googleJavaFormat is configured in build.gradle. This indicates formatting is enforced by the project via Spotless (no explicit task invocation shown in repository docs).
- Tests and CI:
    - GitHub Actions workflow (.github/workflows/gradle.yml) sets up JDK 11 and runs ./gradlew clean test. Use the same commands locally for parity with CI.
- Branching, code review, commit conventions, communication channels and other collaboration processes are not specified in the repository.

License

The repository includes a LICENSE file. The license text in the repository is the MIT License.

Contact Information

Project Maintainer: Not specified in repository metadata.